

Лабораторна робота №3

Розробка серверної частини (Back-end) та API– DnD Companion

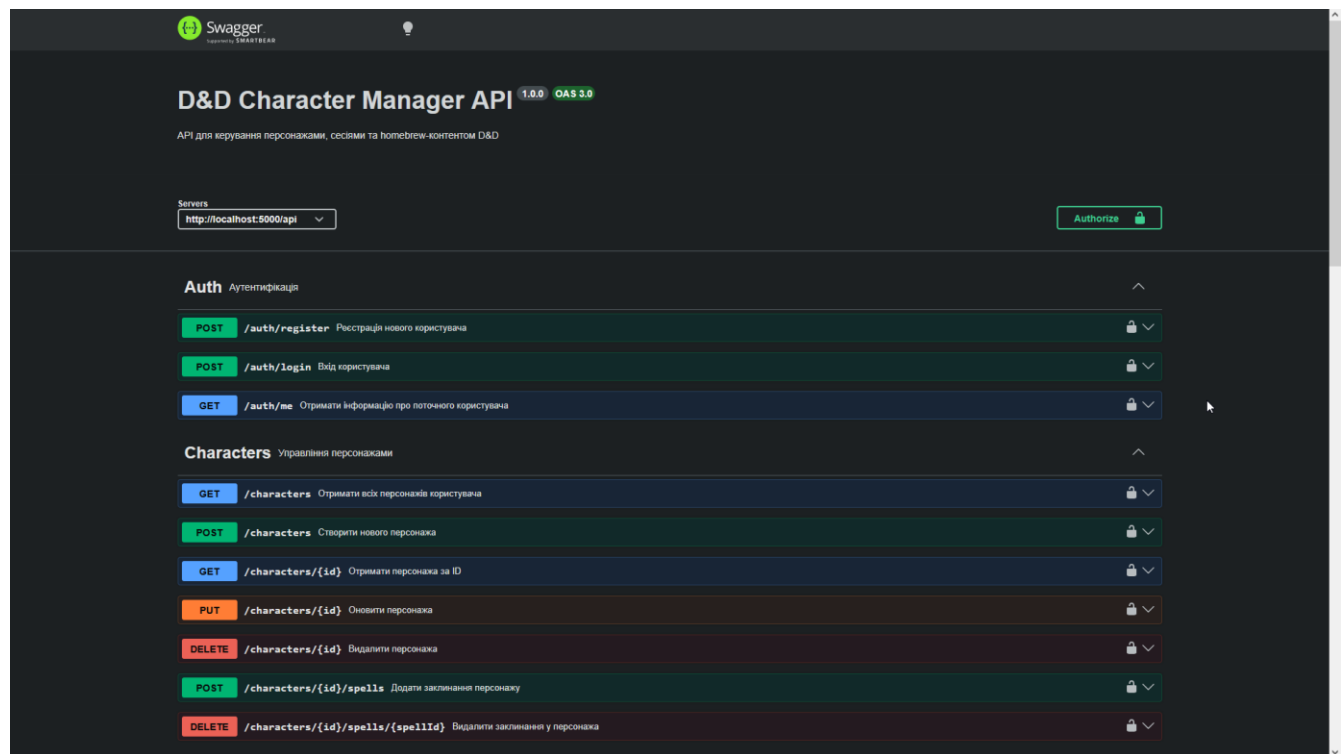
Посилання на Git-репозиторій:

https://github.com/So1oDi/TRVD_2026_402_TN_Zahorovski_Labs

Скріншоти роботи API:

Swagger UI

Всі ендпоінти:



Homebrew Кастомний контент (раси, класи, заклинання, предмети)

GET	/homebrew/races	Отримати всі раси (стандартні + homebrew)	🔒
POST	/homebrew/races	Створити нову homebrew расу	🔒
GET	/homebrew/races/{id}	Отримати расу за ID	🔒
PUT	/homebrew/races/{id}	Оновити homebrew расу	🔒
DELETE	/homebrew/races/{id}	Видалити homebrew расу	🔒
GET	/homebrew/classes	Отримати всі класи (стандартні + homebrew)	🔒
POST	/homebrew/classes	Створити новий homebrew клас	🔒
GET	/homebrew/classes/{id}	Отримати клас за ID	🔒
PUT	/homebrew/classes/{id}	Оновити homebrew клас	🔒
DELETE	/homebrew/classes/{id}	Видалити homebrew клас	🔒
GET	/homebrew/spells	Отримати всі заклинання (стандартні + homebrew)	🔒
POST	/homebrew/spells	Створити нове homebrew заклинання	🔒
GET	/homebrew/spells/{id}	Отримати заклинання за ID	🔒
PUT	/homebrew/spells/{id}	Оновити homebrew заклинання	🔒
DELETE	/homebrew/spells/{id}	Видалити homebrew заклинання	🔒
GET	/homebrew/items	Отримати всі предмети (стандартні + homebrew)	🔒
POST	/homebrew/items	Створити новий homebrew предмет	🔒
GET	/homebrew/items/{id}	Отримати предмет за ID	🔒

PUT	/homebrew/items/{id}	Оновити homebrew предмет	🔒
DELETE	/homebrew/items/{id}	Видалити homebrew предмет	🔒

Sessions Управління ігровими сесіями

GET	/sessions	Отримати всі сесії користувача (да DM або учасник)	🔒
POST	/sessions	Створити нову сесію (DM)	🔒
GET	/sessions/{id}	Отримати сесію за ID	🔒
POST	/sessions/join	Приєднатися до сесії за інвайтом	🔒
POST	/sessions/{id}/leave	Покінити сесію (не DM)	🔒
DELETE	/sessions/{id}/close	Закрити сесію (тільки DM)	🔒
GET	/sessions/{id}/characters	Отримати всіх персонажів у сесії	🔒

Schemas

CharacterResponse >

SessionResponse >

RaceResponse >

Auth Аутентифікація

POST

/auth/register Реєстрація нового користувача

```
curl -X 'POST' \
  'http://localhost:5000/api/auth/register' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "email": "daniilzagarovskiy@gmail.com",
    "password": "qwerty123",
    "display_name": "SoloDi",
    "role": "admin"
  }'
```

Request URL

```
http://localhost:5000/api/auth/register
```

Server response

Code	Details
------	---------

201

Response body

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCIGl0YmNTQWMTI3LWlnZWZhbnNDE5ZS11bzFhLTkyMDMyYWRLITQ3MjYsImVtYSUiOiJoIjZGUwIiwiaWF0IjE5OTYxODkzMjcwLnBvcyvdnBraXcxQDQtYmlsLmVibSI6InVjbGVUIHJhZCpbIjoiImhhbmRTc3NTE2NTlybykiLCJmoxnci6ImJxOXMjIiwicf_t_dffgHL4SY3cv7beu8dqd-6xrHTdtldq63SAmsks",
  "user": {
    "id": "52540127-bfed-419e-b71a-92032adee473",
    "email": "danilzagarovskiy@gmail.com",
    "display_name": "SoloDI",
    "role": "admin"
  }
}
```

Download

GET

/auth/me Отримати інформацію про поточного користувача

```
curl -X 'GET' \
      'http://localhost:5000/api/auth/me' \
      -H 'accept: */*'`
```

Request URL

```
http://localhost:5000/api/auth/me
```

Server response

Code	Details
------	---------

401

Error: Unauthorized

```
Response body
{
  "message": "No token provided"
}
```

POST

/auth/login Вхід користувача

```
curl -X 'POST' \
'http://localhost:5000/api/auth/login' \
-H 'accept: */*' \
-H 'Content-Type: application/json' \
-d '{
  "email": "daniilzagarovskiy1@gmail.com",
  "password": "qwerty123"
}'
```

Request URL

`http://localhost:5000/api/auth/login`

Server response

Code	Details
------	---------

200

Response body

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6ImVudmQ0MTNDDE5ZS11bzFhLkkyMDMyYWRlZTQ3MjYyImVtYWlsIjoizGFuYmluemFnYXQvdnNraXcxQGdtYWlsbmVkbSI6bnVjbGU0Ij3ZGlpbiIsImhhcmWmtc3MTE2NTYxOStwZWxzZXNjdXoxNmciOmVudE5fQm.N3g8bVVUnYMsAlK3Fu3B36IT3H6L-mB3vm4Cxiq",
  "user": {
    "id": "52540127-bfed-419e-b71a-92032adee473",
    "email": "daniil.zagorovskiy@gmail.com",
    "display_name": "SoloDI",
    "role": "admin"
  }
}
```

 Download

ETA

[illegible]Req

1

```
curl -X 'GET' \
  'http://localhost:5000/api/homebrew/races' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjYyNTQwMTI3LWZmZWQtdGE5ZS11bnZFLTKyMDMyYmRlZTQ3MyIsImVtYWliSIjoIjZGFuaWIsZW50bSIsInJvbGU0IjZG1pb1IsIm1hdCI6MTc
```

Request URL

http://localhost:5000/api/homebrew/races

Server response

Code	Details
200	Response body <pre>[]</pre>

Download

POST /homebrew/races Створити нову homebrew расу

```
curl -X 'POST' \
  'http://localhost:5000/api/homebrew/races' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjYyNTQwMTI3LWZmZWQtdGE5ZS11bnZFLTKyMDMyYmRlZTQ3MyIsImVtYWliSIjoIjZGFuaWIsZW50bSIsInJvbGU0IjZG1pb1IsIm1hdCI6MTc' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "Elf",
    "speed": 30,
    "traits": {
      "darkvision": true
    }
  }'
```

Request URL

http://localhost:5000/api/homebrew/races

Server response

Code	Details
201	Response body <pre>{ "id": "2f5e58d9-b3e6-4863-aaF5-f6a808119f84", "name": "Elf", "speed": 30, "traits": { "darkvision": true }, "is_homebrew": true, "creator_id": "52540127-bfed-419e-b71a-92032adee473" }</pre>

Download

Homebrew Кастомний контент (раси, класи, заклинання, предмети)

GET /homebrew/races Отримати всі раси (стандартні + homebrew)

```
curl -X 'GET' \
  'http://localhost:5000/api/homebrew/races' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjYyNTQwMTI3LWZmZWQtdGE5ZS11bnZFLTKyMDMyYmRlZTQ3MyIsImVtYWliSIjoIjZGFuaWIsZW50bSIsInJvbGU0IjZG1pb1IsIm1hdCI6MTc
```

Request URL

http://localhost:5000/api/homebrew/races

Server response

Code	Details
200	Response body <pre>[{ "id": "2f5e58d9-b3e6-4863-aaF5-f6a808119f84", "name": "Elf", "speed": 30, "traits": { "darkvision": true }, "is_homebrew": true, "creator_id": "52540127-bfed-419e-b71a-92032adee473" }]</pre>

Download

POST /homebrew/classes Створити новий homebrew клас

POST [/homebrew/spells](#) Створити нове homebrew заклинання

POST /characters Створити нового персонажа

Characters Управління персонажами

GET

/characters Отримати всіх персонажів користувача

The screenshot displays a REST client interface with the following details:

- Request URL:** `http://localhost:5000/api/characters`
- Server response:** A JSON object representing a character.
- Response body:**

```
[
  {
    "id": "d3653bf4-db3b-4782-b651-4ce93ee9375e",
    "name": "Andreas XII",
    "level": 5,
    "current_hp": 42,
    "max_hp": 42,
    "armor_class": 15,
    "stats": {
      "cha": 12,
      "con": 16,
      "dex": 16,
      "int": 18,
      "str": 10,
      "wis": 15
    },
    "experience_points": 0,
    "race_name": "Elf",
    "class_name": "Wizard",
    "user_id": "52540127-bfed-419e-b71a-92032adee473",
    "session_id": null
  }
]
```

A download button is visible at the bottom right of the response area.

GET

/characters/{id} Отримати персонажа за ID

```
curl -X 'GET' \
'http://localhost:5000/api/characters/d3653bf4-db3b-4782-b651-4ce93ee9375e' \
-H 'accept: */*' \
-H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjYyNTQwMTI3LWZmZWQNdE5lSiJnZFlTKyMOMyYWR1ZTQ3MyIsImVtYWlsIjoiazGFuaWIsbnFhYXNvdnNraXkxQGdtYWlsbmVybSIsInJvbmGUiozOG1pbGlTsImhdCI6MTc
```

<

Request URL

<http://localhost:5000/api/characters/d3653bf4-db3b-4782-b651-4ce93ee9375e>

Server response

Code	Details
200	Response body <pre>{ "id": "d3653bf4-db3b-4782-b651-4ce93ee9375e", "name": "Andreas XII", "level": 5, "current_hp": 42, "max_hp": 42, "armor_class": 15, "stats": { "cha": 12, "con": 16, "dex": 14, "int": 18, "str": 10, "wis": 15 }, "experience_points": 0, "race_name": "Elf", "class_name": "Wizard", "user_id": "52540127-bfed-419e-b71a-92032adee473", "session_id": null }</pre>

Download

PUT

/characters/{id} Оновити персонажа

```
{
  "id": "d3653bf4-db3b-4782-b651-4ce93ee9375e",
  "name": "Andreas XII",
  "level": 5,
  "current_hp": 33,
  "max_hp": 49,
  "armor_class": 15,
  "stats": {
    "cha": 12,
    "con": 16,
    "dex": 14,
    "int": 18,
    "str": 10,
    "wis": 15
  },
  "experience_points": 8,
  "user_id": "525d08127-bfed-419e-b71a-92832adee473",
  "session_id": null
}
```

/characters/{id}/spells Додати заклинання персонажу

```
{
  "message": "Заклиняння додано"
}
```

```
{
  "id": "d3653bf4-db3b-4782-b651-4ce93ee9375e",
  "name": "Andreas XII",
  "level": 5,
  "current_hp": 33,
  "max_hp": 42,
  "armor_class": 15,
  "stats": {
    "cha": 12,
    "con": 16,
    "dex": 14,
    "int": 18,
    "str": 10,
    "wis": 15
  },
  "experience_points": 8,
  "race_name": "Elf",
  "class_name": "Wizard",
  "user_id": "52540127-bfed-419e-b71a-92032adee473",
  "session_id": null
}
```


DELETE**/characters/{id}/spells/{spellId}** Видалити заклинання у персонажа

```
curl -X 'DELETE' \
  'http://localhost:5000/api/characters/d3653bf4-db3b-4782-b651-4ce93ee9375e/spells/f00b39d8-3507-4eef-9ca9-cf599d536e6f' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjUyNTQwMTI3LWZmZWQtdmV5ZS51InZFLlR5bGVyYm91ZTQ3MyIsInVtYWliSIjoiIjZGfuaWJsemFnYXJvdnNraXkxQGdtYWlsLmV5bSIsInJvbnUiOiJhZG1pb1IsIm1hdCI6MTc
```

Request URL

http://localhost:5000/api/characters/d3653bf4-db3b-4782-b651-4ce93ee9375e/spells/f00b39d8-3507-4eef-9ca9-cf599d536e6f

Server response

Code	Details
204	Response headers <pre>access-control-allow-origin: * connection: keep-alive content-security-policy: default-src 'self';base-uri 'self';font-src 'self' https: data;;form-action 'self';frame-ancestors 'self';img-src 'self' data;object-src 'none';script-src 'self';script-src-attr 'none';style-src 'self' https: 'unsafe-inline';upgrade-insecure-requests cross-origin-opener-policy: same-origin cross-origin-resource-policy: same-origin date: Thu,02 Apr 2026 21:55:21 GMT keep-alive: timeout=5 origin-agent-cluster: ?1 referrer-policy: no-referrer strict-transport-security: max-age=31536000; includeSubDomains x-content-type-options: nosniff x-dns-prefetch-control: off x-download-options: noopen x-frame-options: SAMEORIGIN x-permitted-cross-domain-policies: none x-xss-protection: 0</pre>

DELETE**/characters/{id}** Видалити персонажа

```
curl -X 'DELETE' \
  'http://localhost:5000/api/characters/d3653bf4-db3b-4782-b651-4ce93ee9375e' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjUyNTQwMTI3LWZmZWQtdmV5ZS51InZFLlR5bGVyYm91ZTQ3MyIsInVtYWliSIjoiIjZGfuaWJsemFnYXJvdnNraXkxQGdtYWlsLmV5bSIsInJvbnUiOiJhZG1pb1IsIm1hdCI6MTc
```

Request URL

http://localhost:5000/api/characters/d3653bf4-db3b-4782-b651-4ce93ee9375e

Server response

Code	Details
204	Response headers <pre>access-control-allow-origin: * connection: keep-alive content-security-policy: default-src 'self';base-uri 'self';font-src 'self' https: data;;form-action 'self';frame-ancestors 'self';img-src 'self' data;object-src 'none';script-src 'self';script-src-attr 'none';style-src 'self' https: 'unsafe-inline';upgrade-insecure-requests cross-origin-opener-policy: same-origin cross-origin-resource-policy: same-origin date: Thu,02 Apr 2026 21:55:54 GMT keep-alive: timeout=5 origin-agent-cluster: ?1 referrer-policy: no-referrer strict-transport-security: max-age=31536000; includeSubDomains x-content-type-options: nosniff x-dns-prefetch-control: off x-download-options: noopen x-frame-options: SAMEORIGIN x-permitted-cross-domain-policies: none x-xss-protection: 0</pre>

Приклади коду:

DTO для створення персонажа (CreateCharacterDto):

```
export class CreateCharacterDto {
  @IsString()
  name: string;

  @IsOptional()
  @IsUUID()
  race_id?: string;

  @IsOptional()
  @IsUUID()
  class_id?: string;

  @IsInt()
```

```

@Min(1)
level: number = 1;

@IsOptional()
@IsInt()
@Min(0)
experience_points?: number;

@IsInt()
@Min(1)
current_hp: number;

@IsInt()
@Min(1)
max_hp: number;

@IsOptional()
@IsInt()
@Min(8)
armor_class?: number;

@IsObject()
stats: Record<string, number>;

@IsOptional()
@IsUUID()
session_id?: string;
}

```

DTO для відповіді (CharacterResponseDto)

```

export class CharacterResponseDto {
  id: string;
  name: string;
  level: number;
  current_hp: number;
  max_hp: number;
  armor_class: number;
  stats: any;
  experience_points: number;
  race_name?: string;
  class_name?: string;
  user_id: string;
  session_id?: string;

  constructor(character: any) {
    this.id = character.id;
    this.name = character.name;
    this.level = character.level;
    this.current_hp = character.current_hp;
    this.max_hp = character.max_hp;
    this.armor_class = character.armor_class;
    this.stats = character.stats;
    this.experience_points = character.experience_points;
    this.race_name = character.race?.name;
    this.class_name = character.class?.name;
    this.user_id = character.user_id;
    this.session_id = character.session_id;
  }
}

```

Контролер CharacterController з реалізацією Swagger-y:

```
/**
 * @swagger
 * /characters:
 *   post:
 *     summary: Створити нового персонажа
 *     tags: [Characters]
 *     security:
 *       - bearerAuth: []
 *     requestBody:
 *       required: true
 *       content:
 *         application/json:
 *           schema:
 *             type: object
 *             required: [name, current_hp, max_hp, stats]
 *             properties:
 *               name:
 *                 type: string
 *               race_id:
 *                 type: string
 *               class_id:
 *                 type: string
 *               level:
 *                 type: number
 *               current_hp:
 *                 type: number
 *               max_hp:
 *                 type: number
 *               armor_class:
 *                 type: number
 *               stats:
 *                 type: object
 *               session_id:
 *                 type: string
 *     responses:
 *       201:
 *         description: Персонаж створений
 */
async createCharacter(req: Request, res: Response) {
  const userId = req.user!.id;
  const dto = req.body as CreateCharacterDto;
  const newChar = await this.characterService.createCharacter(userId, dto);
  res.status(201).json(new CharacterResponseDto(newChar));
}
```

Відповіді на контрольні запитання

1. Порівняйте REST, GraphQL та gRPC. У яких випадках доцільно використовувати кожен із них?

REST – архітектурний стиль на основі HTTP, де кожен ресурс має свій URL (/characters, /sessions). Простий, добре знайомий, підтримується всіма клієнтами. Доцільний для публічних API, CRUD-орієнтованих сервісів і випадків коли важлива широка сумісність.

GraphQL – мова запитів, де клієнт сам описує які поля йому потрібні. Один ендпоінт `/graphql` замість багатьох. Доцільний коли різні клієнти (мобільний, веб) потребують різних наборів даних з одного бекенду, або коли дані сильно пов'язані між собою.

gRPC – фреймворк від Google на основі HTTP/2 та Protobuf. Дуже швидкий, типізований, підтримує стримінг. Доцільний для внутрішньої міжсервісної комунікації (мікросервіси), де важлива максимальна продуктивність і строга типізація.

2. Що таке проблема Over-fetching та Under-fetching даних? Як GraphQL вирішує ці проблеми порівняно з REST?

Over-fetching – сервер повертає більше даних ніж потрібно клієнту. Наприклад, запит `GET /characters/1` повертає весь об'єкт персонажа з 20 полів, хоча на екрані потрібні лише ім'я та HP.

Under-fetching – одного запиту недостатньо і доводиться робити кілька. Наприклад, щоб показати картку персонажа з назвою раси, потрібно спочатку отримати персонажа, потім окремо запитати расу по `race_id`.

GraphQL вирішує обидві проблеми: клієнт в одному запиті точно вказує які поля потрібні і може "провалитися" в пов'язані сутності.

3. Що таке Protocol Buffers (Protobuf)? Які його переваги перед JSON?

Protobuf – бінарний формат серіалізації даних від Google. Структура даних описується в `.proto` файлі, з якого автоматично генерується код для потрібної мови.

Переваги перед JSON:

- **Розмір** – бінарне повідомлення в 3–10 разів менше за JSON, бо не зберігає назви полів у кожному повідомленні.
- **Швидкість** – серіалізація/десеріалізація значно швидша ніж парсинг текстового JSON.

- **Строга типізація** – типи задані в схемі, помилки типів виявляються на етапі компіляції.
- **Автогенерація коду** – з одного .proto файлу генерується клієнт і сервер для десятків мов.

Недолік – бінарний формат нечитабельний для людини без спеціальних інструментів, що ускладнює дебагінг.

4. Як забезпечується версіонування API у REST та GraphQL?

У **REST** найпоширеніший спосіб – версія в URL: /api/v1/characters, /api/v2/characters. Альтернативи – версія в заголовку (Accept: application/vnd.api+json;version=2) або в query-параметрі (?version=2). URL-версіонування є найпростішим і найзрозумілішим.

У **GraphQL** версіонування в URL зазвичай не практикується. Замість цього використовується підхід "**еволюція схеми**": старі поля не видаляються, а позначаються як застарілі через директиву @deprecated(reason: "Use newField instead"). Нові поля додаються поряд зі старими. Клієнти мігрують поступово без breaking changes.

5. Поясніть концепцію Idempotency (ідемпотентності). Які методи/операції мають бути ідемпотентними?

Операція є ідемпотентною якщо її повторне виконання з тими самими параметрами дає той самий результат що й перше виконання. Тобто 1 виклик = 10 викликів з точки зору стану системи.

Метод	Ідемпотентний?	Пояснення
GET	Так	Читання не змінює стан
PUT	Так	Повна заміна ресурсу – завжди той самий результат
DELETE	Так	Після першого видалення повторний DELETE не змінює стан (ресурс вже відсутній)

PATCH	Залежить	{ currentHp: 30 } – ідемпотентний; { delta: -5 } – ні
POST	Ні	Кожен виклик створює новий ресурс

Ідемпотентність важлива для retry-логіки: якщо запит не дійшов через мережевий збій, клієнт може безпечно повторити його не боячись дублікатів.

6. Яка роль Schema Definition Language (SDL) у GraphQL або *.proto* файлів у gRPC?

Обидва є **мовами опису контракту API** – тобто формальним описом того, які дані та операції доступні, незалежно від реалізації.

SDL (Schema Definition Language) у GraphQL описує типи даних, запити (Query) та мутації (Mutation). Слугує одночасно документацією, основою для автоматичної валідації запитів і джерелом для генерації типів на клієнті (наприклад, через GraphQL Code Generator). Без схеми GraphQL-сервер не може працювати.

.proto файли у gRPC описують сервіси, методи та структури повідомлень. Є єдиним джерелом істини: з одного .proto файлу генерується код клієнта і сервера для будь-якої підтримуваної мови (Go, Java, Python, TypeScript тощо). Це гарантує що клієнт і сервер завжди говорять "однією мовою".